Repowire: A Persistent Mesh for Heterogeneous Coding Agents

Prasanna Ravishankar [1] [co-author line]

Abstract

Coding agents are session-bound: an agent exists for the duration of one terminal session, holds one repository in context, and is coordinated with other agents by the person who operates them. When work spans repositories and runtimes, that person becomes the message bus. We present Repowire, a local-first daemon that turns coding-agent sessions on several runtimes (Claude Code, Codex, OpenCode, Pi) into addressable, persistent peers of one mesh. The design rests on four decisions: a peer identity that is minted by the daemon, proved by process evidence, and separated from both display name and runtime session; a four-message protocol (`ask`, `ack`, `notify`, `broadcast`) in which only asks carry a lifecycle; runtime-specific bridges beneath a transport-neutral router; and lazy repair in place of heartbeats or watchdogs. Humans join the mesh as peers through a phone bot and a dashboard, and an orchestrator is an ordinary peer with a coordinating role. We evaluate the system from its own event journal: 146 days of production use by one operator, 2,810 requests, 370 agent names over 86 projects, six runtimes, and 76 written incidents. The journal shows that identity outlives sessions (one project name spans 144 days over 49 registrations while the median registration lives 12.6 minutes), that request closure rose from 87.4% to 98.1% after identity and delivery were made durable, that 52% of agent-to-agent requests cross runtimes, that 75% of requests repeat the sender's previous recipient, and that human surfaces originate 54 to 60% of non-machine requests and no notifications. Coding the incident record against an existing failure taxonomy for multi-agent systems locates 34 of 76 failures in a class that single-run traces cannot exhibit: lifecycle. Repowire is open source.

[1]Independent [affiliation]. Correspondence to: Prasanna Ravishankar <me@prassanna.io>.

Preprint. Draft of 14 October 2026. Under preparation for submission.

1. Introduction

A coding agent that holds one repository in context outperforms one that holds five, for the same reason a specialist engineer does (Ravishankar, 2026b). The cost of specialisation is coordination. When a task spans a frontend, a backend, and the infrastructure that deploys them, each agent sees its own repository and none of the others, and the person running them relays questions and answers between terminals. The relay is slow and lossy: summaries drop flags, version numbers, and the detail that would have explained the error. Every existing remedy is push-based. Documentation, memory banks, ticket summaries and shared super-folders publish context ahead of need and go stale on the next commit.

The alternative is pull. An agent that needs to know the shape of an API should ask the agent that is looking at that API now. That requires three things no coding-agent runtime provides: a stable address for a session that may be restarted, resumed, or replaced; a delivery path into a running session on whichever runtime hosts it; and a record of what was asked, of whom, and whether it was answered.

This paper describes Repowire, a system that provides them, and measures what happened when one operator ran it for five months. Repowire is a daemon on the developer's machine. Agent sessions register with it as *peers*; the daemon routes addressed messages among them, into the native input of each runtime, and records every message in a journal. Humans are peers too: a Telegram bot, a Slack bot and a web dashboard register with the `human` role and use the same four message types as agents. An *orchestrator* is a peer with a coordinating role and no special privileges in the router.

The design was not derived from a model of multi-agent coordination. It was derived from incidents. The first version (January 2026) routed by display name and terminal pane; the five months of production use that followed produced a written record of 76 failures, most of which concern identity, liveness, and delivery rather than reasoning. The design decisions this paper defends are the ones those failures forced.

Contributions.

1. A design for persistent, addressable coding agents

across runtimes (Section 3): daemon-minted identity with a proof hierarchy and terminal retirement; a protocol in which only asks have a lifecycle; bridges under a transport-neutral router; lazy repair instead of liveness polling; humans and orchestrators as peers.

2. An open-source implementation (Section 4) that grew from 5 to 33 tools and from one runtime to six over ten months, with a native cutover that makes the journal itself a measurement instrument.

3. A measurement study of the mesh from its journal (Section 5): 146 days, 2,810 requests, 2,472 agent registrations. We report lifetime and identity churn, closure and latency, the sender-recipient structure of traffic including the human's share, cross-runtime use, the review loop, and incidents coded against the MAST taxonomy (Cemri et al., 2025).

4. Design lessons (Section 6) stated as claims with the evidence that supports or limits them.

The study is of one mesh with one operator, and the paper says so wherever a number depends on it (Section 8). Its value is the inside view: who asked whom, how long closure took, what died and when. Prior measurement work observes coding agents from the repository (Agarwal et al., 2026; Mazloomzadeh et al., 2026) or by practitioner survey (Pan et al., 2026); benchmarks score task-spawned agents that do not outlive the task (Khatua et al., 2026). A persistent mesh with a full journal has not previously been reported.

2. Setting and Goals

The operator's situation. One developer maintains several repositories (a product, its infrastructure, a few libraries, a website) and runs a coding agent in each. The agents are on different runtimes because the runtimes have different strengths and separate usage quotas. Work arrives from a phone, a laptop, or a scheduled job. Sessions are long-lived in intent and short-lived in practice: terminals close, laptops sleep, runtimes are upgraded, daemons restart.

Goals.

G1 Address. A session has a stable address that survives reconnects and daemon restarts, and that no other session can take by sharing a path or a name.

G2 Deliver. A message to a live session reaches its native input, and the sender learns whether it did.

G3 Heterogeneity. Runtimes differ in hooks, session models and input paths; the protocol must not.

G4 Humans as peers. The person is addressable and addressed by the same primitives as agents, from a phone.

G5 No polling. An idle mesh costs nothing; agents do not heartbeat.

G6 Record. Every request, reply, registration and repair is journaled so that the mesh can be debugged and, later, measured.

G5 is the unusual one. It was adopted because every polling loop in the first versions became an incident source, and it has a measurable cost in liveness latency (Section 3.5).

3. Design

Figure 1 shows the components. The daemon is the only routing hub. Everything else, including human surfaces and the hosted relay, is a client.

3.1. Peers, circles, roles

A *peer* is one running agent session: a display name derived from its working directory and runtime (`torale-codex`, suffixed on collision to `torale-2-codex`), a working path, a runtime (*backend*), a status (online, busy, offline), a per-turn state, a free-text description the agent sets, and a last-seen time. A *circle* is a routing scope that by default maps to a terminal multiplexer session, so agents started together can address each other and agents started elsewhere cannot. A *role* is metadata: `agent` by default; `human` for control surfaces; `orchestrator` for a coordinating session; `service` for the job runner and the relay. Roles that bypass circles may name a circle explicitly; an ordinary agent cannot.

3.2. Identity

The central design decision is that the routing key is a daemon-minted `peer_id`, not the display name, not the working path, and not the runtime's own session id. Each of the alternatives caused incidents (Section 5.6): names collide across circles, paths are shared by sibling terminals and inherited by subprocesses, and runtime session ids are reused on resume and absent on some runtimes.

Proof hierarchy. A registration or reconnect claiming an identity is resolved against evidence in a fixed order (Table 1). A live process id on the same machine, runtime and compatible path is the strongest evidence a registrant carries: several registrants describe the same agent independently (a session-start hook, the tool server's lazy registration, a socket reconnect), and all of them converge on the peer already registered for that process. A short-lived *birth certificate* minted at registration lets the tool server adopt the identity without a path search. Path and name are hints only; they filter and allocate names but never prove identity, and an ambiguous match is a refusal, not a guess.

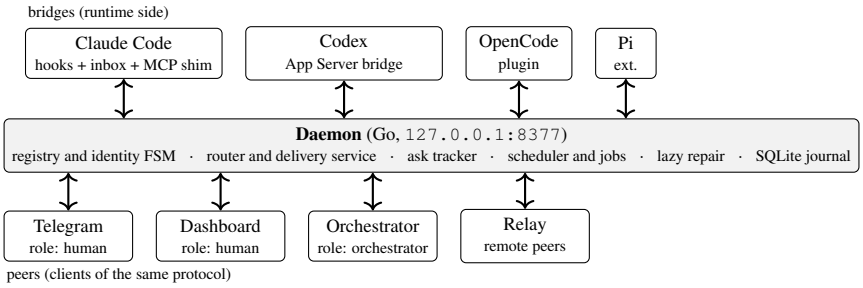
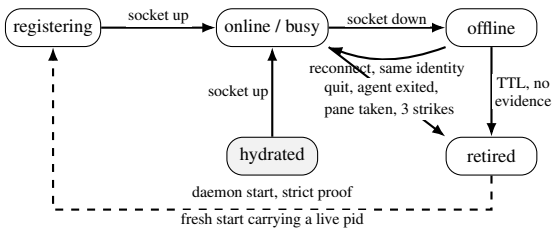


Figure 1. Components. Bridges translate each runtime's native session API into the peer protocol; the daemon routes at the peer level and never sees runtime detail. Human surfaces, the orchestrator and the relay are clients.

Table 1. Identity proof hierarchy. Routing resolves from the strongest available level and fails closed below level 5.

Level	Evidence	Authorises
1	live `peer_id` in the registry	routing
2	daemon-minted birth certificate	identity handoff to the tool server
3	pane metadata plus live process id	local adoption
4	durable session mapping	circle, role, history provenance
5	spawn ownership or pane metadata naming the `peer_id`	kill and restart
6	name, path, runtime, machine, last seen	hints and filters only



Lifecycle. Section 3.2 shows the states. The transitions that matter are the terminal ones. A clean quit, a hook reporting that its agent process exited, a pane takeover, or three consecutive honest negative liveness verdicts *retire* the identity: the daemon severs its socket and refuses any reconnect claiming the id unless the claim carries a live process id. Retirement records persist across daemon restarts, so an orphaned reconnect loop cannot resurrect a dead peer. A fresh session start always carries a live process id and reclaims the name cleanly. Without retirement, a dead session's supervisor reconnects forever and the registry fills with phantoms; this was the most frequent incident class before retirement existed.

Restart. On daemon start, a one-shot *hydration* pass recreates peers for panes that were alive while the daemon was down, but only when daemon-written pane metadata names the same `peer_id` and the runtime, normalised path, live pane working directory and live process id all agree. Hydrated peers are offline until a socket connects; the daemon does not assume delivery it cannot observe. Unproven panes stay unadopted and are listed for explicit linking by the operator.

Descriptions and sessions. A peer's *description* is agent-set task state with no expiry; each turn is shown the current value and asked to update it. A *session* is the durable context behind a peer: a binding from the daemon's session id to the runtime's own session id, which permits resume after the executor is gone. Peers route; sessions persist.

3.3. Message protocol

The daemon routes four message types, chosen by lifecycle rather than content.

- `ask(to, text)` is non-blocking and returns a correlation id. The recipient closes it with `ack(id)` (seen), `ack(id, reply)` (reply delivered to the asker), or `decline(id, reason)`. Follow-ups chain with `ask(reply_to=id)`. Until it is closed, the recipient sees a reminder at the start of every subsequent turn. Only a real tool call closes an ask; prose mentions do not.
- `notify(to, text)` is fire-and-forget with no lifecycle.
- `broadcast(text)` fans out within the sender's circle.
- A *structured question* is an ask with a typed envelope (choice, text, tool permission) answered through `answer`; tool-permission questions deny on timeout.

Peer-originated deliveries are framed so that a runtime can distinguish them from its user's instructions; messages from human surfaces are framed as user instructions. An ask to a peer the runtime says cannot accept direct input (a sub-agent

thread) is refused before any state is created. Replies always reach the asker regardless of circle, because the route was fixed at ask time.

The asymmetry between `ask` and `notify` is deliberate. Giving every message a lifecycle produces open threads nobody owns; giving none produces work that disappears. The journal shows the choice is used as intended: humans send asks and never notifications; agents send both (Section 5.4).

3.4. Bridges and transports

A *bridge* is the runtime-side component that speaks a runtime's native session API on one side and the peer protocol on the other. Bridges share a wire contract, not an implementation or a process model: Claude Code uses lifecycle hooks, a per-session authenticated inbox for inbound delivery, and a stdio identity shim that proxies tool calls to the daemon; Codex uses a separately supervised companion on the App Server socket that registers threads as peers before their first prompt and steers active turns; OpenCode and Pi load a plugin or extension inside the runtime. A *transport* is the delivery path a bridge exposes, with truthful receipts: a message that cannot be injected is reported as failed, never silently queued behind a keystroke fallback. The router addresses peers and sessions, never panes, hook files or sockets, so adding a runtime adds a bridge and nothing in routing.

3.5. Lazy repair

There are no heartbeats, no background polls, and no watchdog thread. Liveness checks, persistence flushes and ghost eviction run at most once per 30 seconds and only while a tool call is already being handled. Clean quits deregister through a session-end hook; a crashed agent is reported by its own socket supervisor within seconds; repair catches what those tiers miss. The consequence is that peer lists are eventually consistent: a session that crashed without a hook can show online for up to a minute. The daemon makes the inconsistency visible rather than hiding it, emitting a `peer_contradiction` event once per transition (online without a socket, pane missing, agent process dead), and offers an operator-triggered reconciliation that reports identity, reachability, open asks and contradictions on demand. The 30-second bound was tuned against a self-inflicted failure mode: the one timer-driven reconciliation in the first versions mass-offlined 43 peers on a spurious signal (Section 5.6).

3.6. Humans and orchestrators as peers

Control surfaces register with the `human` role, bypass circles, and use the same four message types. A message from the phone arrives at an agent as a user instruction; an agent's reply to the phone is an `ack` with text. Structured questions render as buttons. A *human inbox* (October 2026) queues asks addressed to the human across surfaces.

An orchestrator is a peer with `role=orchestrator`; the daemon enforces nothing about it. It is a long-running session the operator addresses from a phone, which scans a board, dispatches asks or durable jobs to project peers, tracks replies, requests reviews, and closes out. Its workspace holds communication preferences, project scope, and memory files; the daemon performs a bounded lexical recall over them and prefixes matching entries to inbound messages. Role is data: the daemon reads the field, and a second orchestrator can coexist as an observer.

3.7. Schedules, jobs, and the journal

A *schedule* delivers a future message (notification or tracked ask). A *job* is a durable work record with its own executor: for a job fire, the daemon dispatches an ask from a built-in `@jobs` peer, arms the fire when the executor's turn carrying the marker begins, and completes it from the executor's next assistant turn, so results are captured structurally rather than self-reported. An executor that dies with a fire in flight fails it with `executor_died`; a delivered fire is never reaped by a clock.

The journal (G6) is an append-only event table in SQLite plus a per-message delivery-trace ledger recording each stage (created, delivered, closed) with resolved ids and status. It exists so that a misroute can be diagnosed from fields rather than ad hoc logs. Section 5 uses it as the measurement instrument.

4. Implementation

Repowire is a single Go binary (daemon, CLI, hooks, bridges, tool server, Telegram and Slack clients, relay client) with an embedded dashboard; the first eight months were Python, and the native cutover (PR #382, 10 August 2026) splits the journal into two eras with different schemas (Section 5.1). State is SQLite with versioned migrations, which date the journal's own surfaces: event journal (22 May 2026), tracked work (25 May), delivery traces (30 May), retired identities (11 June), peer provenance (18 September), routing decisions (11 October), human inbox (13 October). The tool surface grew from 5 tools in December 2025 (`ask_peer`, `notify_peer`, `broadcast`, `list_peers`, `register_peer`) to 33 in October 2026, with the largest step (8 to 28) in May 2026 when asks gained lifecycle (`ack`, `answer`, `ask_many`), jobs and schedules arrived, and the orchestrator role was introduced. Six runtimes have registered peers; Gemini CLI and Antigravity were later retired as supported backends. A hosted relay tunnels dashboard and socket traffic so that a mesh on a

home machine is operable from elsewhere; remote agents can join through a relay-side tool server. The repository has 828 commits and 362 pull requests from December 2025 to October 2026, all but 18 by the operator's account, and is Apache-licensed. [artifact URL]

5. Evaluation

We ask five questions of the journal. Do identities in fact outlive sessions, and what does a restart do (Section 5.2)? Does the protocol close what it opens, and how fast (Section 5.3)? Who talks to whom, and what does the human do (Section 5.4)? Is heterogeneity used (Section 5.5)? Where do failures come from (Section 5.6)?

5.1. Method and data

The data are a backup of the daemon's state database taken on 14 October 2026 and opened immutable; the daemon log, from which only start markers were extracted; the repository's history and pull-request record; and the operator's written notes (project file, 150 orchestrator memory files, 46 vault notes, two repository memory directories), from which incidents were compiled. Configuration and credential files were not read. The journal spans 22 May to 14 October 2026 (146 days; 93 with any event; 64 with an active agent peer). It changes shape at the native cutover: era A (22 May to 10 August, Python) journals asks as events and replies as notification text, and has no registration events; era B (10 August to 14 October, Go) has registration and status events, typed ack events, and the delivery-trace ledger. Asks are unified across eras (2,810: 1,505 machine dispatch from the job runner, 1,305 other). We report per era where the measure depends on the schema. Sender classes: *human* (Telegram, dashboard, Slack, inbox), *orchestrator* (role or name), *peer* (other agents), *machine* (job runner), *relay* (remote tool server). Scripts and the backup copy are kept with the paper's working files; the backup is not released because ask text contains the operator's private material. Section 8 lists what the journal cannot answer.

Activity. Figure 2 shows monthly volume by sender class. Two regimes are visible: May and June, when the mesh ran many agents and a job runner (925 of June's 1,321 asks are machine dispatch; July is almost entirely machine), and August to October, when the native daemon ran a smaller mesh without jobs. The system was in use throughout; the measures below are reported in a way that does not depend on the volume difference.

5.2. Identity outlives sessions

Lifetimes. A `peer_id` is a registration; one project session can hold several across reconnects, so per-id lifetimes bound session lifetime from below. Table 2 gives the distribution. The median agent registration lives 12.6 minutes and 30% are observed exactly once; the median orchestrator registration lives 16.1 minutes. Job executors, which are spawned per fire, have a median of 35 minutes because a fire is one turn and turns are long.

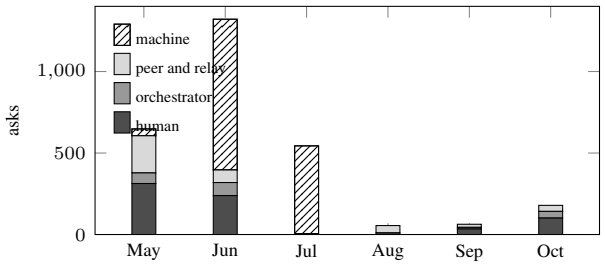


Figure 2. Asks per month by sender class, May to October 2026 (May from the 22nd, October to the 14th). Machine dispatch is the job runner, which was retired at the native cutover.

Table 2. Registration lifetime (last minus first observation of a `peer_id`), by class. Agent and orchestrator lifetimes are lower bounds on session lifetime.

class	n	p25	median	p75	p90
job executor (era A)	1,456	3.1 m	35.0 m	4.4 h	4.8 d
spawned by mesh (era B)	56	9.3 m	11.6 m	38.5 m	5.7 h
other agent	823	0	12.6 m	56.8 m	23.5 h
orchestrator	137	0	16.1 m	2.0 h	37.1 h

Names persist. Display names do not share this churn. Seven names span 138 to 144 days of the 146: `repowire-claude-code` over 49 registrations, `orchestrator-claude-code` over 37, `repowire-codex` over 30, `torale-codex` over 3. The name is what the operator and the other agents address; the id is what the daemon routes. The design in Section 3.2 exists to keep the two consistent under churn of this magnitude: the *same* name must resolve to the *current* registration and never to a dead one, and a dead one must never be resurrected by its own reconnecting supervisor.

Restarts. Daemon restarts are not journaled, but in era B they are dated from the log: 64 starts on 14 days, 58 of them within five minutes of a burst of five or more `peer_online` events, and 41 of 60 such bursts within five minutes of a start. The re-registration storm that follows a restart is the moment identity is tested, and the written record places six multi-incident days there (Section 5.6). In era A, 243 job rows were failed by restart reconciliation at six distinct minutes: a restart killed whatever was in flight.

Contradictions and repair. Lazy repair recorded 428 contradictions across the span (online without a socket 130, pane missing 126, agent process dead 5, uncoded 167) and 485 reaps, all by offline time-to-live. Offline reasons in era

Table 3. Ask closure and acknowledgement latency, non-machine asks, by era.

era	asks	closed	p25	median	p75	p90
A (Python)	1,040	87.4%	14 s	26 s	60 s	2.7 m
B (Go)	265	98.1%	16 s	29 s	66 s	3.1 m

Table 4. Sender and recipient class, non-machine asks (n = 1,305).

sender	recipient	asks	share
human	orchestrator	436	33.4%
peer	peer	348	26.7%
human	peer	262	20.1%
orchestrator	peer	166	12.7%
peer	orchestrator	53	4.1%
orchestrator	orchestrator	24	1.8%
relay	orchestrator	8	0.6%
agent	human	7	0.5%

B are dominated by clean session end (226) and missing socket plus missing pane (115); transport failure accounts for 32. The policy of making inconsistency visible rather than polling it away is thus exercised about three times a day on active days.

5.3. Closure and latency

Of 1,305 non-machine asks, 1,169 have a close record; 136 do not (131 in era A, 5 in era B). Closure rose from 87.4% in era A to 98.1% in era B (Table 3). The era B gains come from three changes made between the eras in response to incidents: retirement (dead recipients are refused at send time rather than accepting into a void), truthful delivery receipts (a failed injection closes the ask with `send_failed`; five such closes in era B), and queued redelivery to a reconnecting orchestrator. Expiry is not journaled, so the open duration of the 136 is unknown.

Latency is time to acknowledgement, which in practice often precedes completion ("on it"); it is not task duration. The median is 26 to 29 seconds in both eras, p90 about three minutes. By sender, the human is acknowledged fastest (median 23 s, p90 105 s), peers in 38 s, the orchestrator in 40 s with a p90 of 8.3 minutes, because orchestrator asks are more often work than questions.

5.4. Who talks to whom

Table 4 gives sender and recipient classes for the 1,305 non-machine asks. Three facts stand out.

The human asks; agents report. Human surfaces originate 53.5% of non-machine asks over the span (52 to 60% in every month with volume), and *no* notifications. The reverse direction is almost absent as asks (7) because agents reach the human through ack replies and notifications to the phone (1,268 orchestrator and peer notifications in May and June alone). The protocol's asymmetry maps onto a behavioural one: the person opens threads and the mesh closes them. This is the operator's stated intent (G4) and the journal confirms it was kept.

The mesh is flat in protocol and hierarchical in use. Nothing in the router privileges the orchestrator, yet 40% of asks are to or from one. Human-to-orchestrator is the single largest flow. Peer-to-peer traffic is substantial (26.7%) and concentrated in the two months when many agents ran, consistent with pull-based cross-repository questions being the original purpose (Section 1); orchestrator-to-peer dispatch grew from 11% of non-machine asks in May to 23% in October as the operator moved to addressing the orchestrator from a phone.

Coordination is habit. Over asks whose sender has a previous ask, 74.7% name the same recipient as that sender's previous ask (0.70 to 0.86 by month). By sender class the share is 0.859 for the human, 0.651 for peers, and 0.423 for the orchestrator. The orchestrator is the only sender that switches often, so recipient selection is an orchestrator problem and not a human one. A companion paper (Ravishankar, 2026a) shows the consequence for evaluating learned routing: on the same journal, a recency rule scores 0.60 overall against a classifier's 0.58, and a frontier model reaches only 0.57 on the switches that are the useful part of the problem.

5.5. Heterogeneity is used

Of 591 agent-to-agent asks, 307 (52%) cross runtimes: Claude Code to Codex 177, Codex to Claude Code 104, Codex to Pi 13, Codex to Antigravity 9. Cross-runtime traffic has a specific use. Reconstructing review traffic from message text, 64 of the 152 repository pull requests opened in the span have at least one review message on the mesh and 37 have a verdict; in 30 of those the requester and reviewer are on different runtimes (23 Claude Code requester with Codex reviewer, 7 the reverse). Of 37 PRs with a verdict, 27 merged after one round, 5 after two, 3 after three, and 2 after five or more. The operator's notes record that a mediated three-round review costs about 12 orchestrator turns and 30k tokens, and that letting peers review each other directly saves about three quarters of it; the protocol's reply-reaches-asker rule is what makes direct peer review possible across circles. The repository's automated review bot produced 714 inline findings on 284 PRs over the same period, 80% of them medium severity; the mesh review is the one with a verdict.

Table 5. Incidents from the written record by class, with the MAST mode they map to when one exists. Counts follow note density and are not a rate.

class	n	MAST mapping
peer leak or phantom identity	13	none (lifecycle)
config or binary drift	10	none (deployment)
delivery drop	8	FM-2.4 withholding, at best
misroute	7	none (lifecycle)
job or schedule failure	7	FM-3.1 premature termination, partly
daemon restart disruption	4	FM-1.4 history loss, partly
auth expiry	2	none (lifecycle)
other	25	4 credential exposures; worktree crossing; stale descriptions; review process; quota
total	76	34 lifecycle or delivery

5.6. Where failures come from

The written record yields 76 operational incidents (dated by note text, not file time; 73 within the study window). We classed them by proximate cause and then coded each against MAST (Cemri et al., 2025), the 14-mode taxonomy derived from 1,642 single-run traces of seven frameworks. Table 5 gives the result.

Thirty-four of the 76 are identity, liveness or delivery failures: a dead session's registration surviving and receiving work (13), a message accepted and lost (8), a message reaching the wrong live session (7), a restart clearing in-flight state (4), a credential lapsing while the peer showed online (2). MAST, built from single-run traces, has no mode for any of these, because in a single run there is no second session for an identity to leak into and no restart to survive. Its nearest modes (loss of conversation history, conversation reset, information withholding) are the single-run shadows of the class. Of the remaining 42, ten are drift between an installed binary or configuration and the source the operator believed was running, a deployment class that also has no MAST mode, and 25 are heterogeneous, including four credential exposures into transcripts or logs in October 2026. Only a handful (a merged PR with review changes unaddressed, test-grinding, mocked validation reported as real) are reasoning or verification failures of the kind MAST catalogues.

Six incidents have a journal signature (a burst of offline, reap or re-registration events within minutes of the note's time), including the timer-driven mass-offline of 43 peers on 21 June 2026 that motivated G5 and the eleven daemon starts on 13 October 2026 during binary drift. The remaining bursts in the journal have no note, and the remaining notes have no burst; neither source is complete.

5.7. Cost

The journal records tokens, not money, and only for the three days of shadow routing at its end. The operator's record gives the order of magnitude: the mesh's hosted relay and web run on a cluster costing about 27 to 37 GBP per month; the frontier-model routing experiments cost about 11 USD for 555 decisions per model; the mesh itself adds no model calls to an agent's turn except the recall prefix and the shadow router. An idle mesh is idle (G5).

6. Lessons

Identity, not intelligence, is where a persistent mesh fails. Thirty-four of 76 incidents and the whole of the era A to era B closure gain are about who a message reaches and whether the recipient is alive. None of the frameworks in MAST's corpus can exhibit these failures because none of their agents outlives a run. Two contemporaneous systems arrived at the same primitive by the same route: Gastown's workers have persistent identity and ephemeral sessions with a watchdog chain (Yegge, 2026), and Claude Code's Agent Teams, which lack resumption of teammates, document the gap as a limitation (Anthropic, 2026). The design claim this paper makes is narrower than "persistent identity is necessary": it is that identity must be *daemon-minted, process-proved and terminally retirable*, and that name, path and runtime session id are each insufficient, with an incident for each.

Only asks need a lifecycle. The ask/notify split was used as designed: humans open, agents close. Reminders on open asks and the rule that only a tool call closes one are what keep closure at 98% without a tracker the operator has to tend.

Lazy repair trades a minute of staleness for the absence of a failure class. Every timer in the first versions became an incident. The cost is that a peer list can be a minute stale and that contradictions must be surfaced rather than resolved. The journal shows about three contradictions per active day, all visible, none silent.

The human is the habit engine. Human senders repeat their previous recipient 86% of the time; the orchestrator 42%. A learned router should therefore sit in front of the orchestrator and be evaluated on switches, not on the pooled stream (Ravishankar, 2026a).

What generalises and what does not. The identity and delivery design generalises to any system where agents outlive runs and are addressed by name. The traffic structure (human share, habit, orchestrator centrality) is one operator's and should be read as an existence proof of a regime,

not a distribution.

7. Related Work

Task-spawned multi-agent frameworks. CAMEL, MetaGPT, ChatDev, AutoGen, CrewAI, Magentic-One and OpenHands (Li et al., 2023; Hong et al., 2024; Qian et al., 2024; Wu et al., 2023; Wang et al., 2024) instantiate agents per task from role templates; agents do not outlive the task, accumulate no history with one another, and are not addressed mid-run by a human. MAST (Cemri et al., 2025) is derived from their traces and, as Section 5.6 shows, has no lifecycle class.

Coding-agent fleets. Claude Code Agent Teams (Anthropic, 2026) run a lead session with teammate sessions over per-agent file mailboxes and a locked task list; teammates are not resumable, one team per session, lead fixed. Gastown (Yegge, 2026) coordinates 20 to 30 Claude Code instances with persistent worker identity, ephemeral sessions, a per-rig lifecycle manager and a background patrol on a three-minute heartbeat, and Wasteland federates many such towns. Cursor's planner-worker design (Cursor, 2026) reports that flat agents over shared files collapsed in throughput. Repowire differs in topology (a flat mesh with an orchestrator that is a peer), in runtime heterogeneity (six runtimes, 52% cross-runtime traffic), in the human being a peer, and in having no heartbeat.

Protocols. MCP is Repowire's tool surface into each runtime; A2A and the interoperability surveys (Ehtesham et al., 2025; Anonymous, 2025) define discovery and task exchange between services but not identity across restarts, delivery receipts, human surfaces, or lifecycle, which is what a working mesh needed.

Measurement. Pan et al. (Pan et al., 2026) measure agents in production across 26 domains by case study and survey (most run at most ten steps before human intervention; reliability the top concern); this paper is the complement, one system from its journal. AIDev-based studies (Agarwal et al., 2026; Mazloomzadeh et al., 2026; Anonymous, 2026c) observe agent pull requests from the repository. CooperBench (Khatua et al., 2026) finds a 30% drop when two coding agents must coordinate and attributes it to cluttered messages, broken commitments and wrong assumptions about the partner; those are failures of the agents, whereas the majority of ours are failures of the substrate beneath them. Routing and delegation benchmarks (Gao et al., 2026; Ren et al., 2026; Hu et al., 2024) are treated in the companion paper.

Persistent state. InfiAgent and the memory characterisation literature (Anonymous, 2026b;a) move state out of the prompt; Repowire's orchestrator memory and session bindings are small instances. A negative result on learned recall over that memory is reported separately.

8. Limitations

One mesh, one operator, who is also the author: traffic structure reflects one person's habits. Incident counts follow note density (May and June have the most notes) and are not a rate. Era A lacks registration events and delivery traces, so lifetimes and closure there rest on activity and parsed reply text (855 of 1,169 close records). Daemon restarts are undated before August. Ask expiry is not journaled, so the 136 unclosed asks have unknown open duration. Acknowledgement is not completion. Agents commit under the operator's identity, so authorship of code cannot be separated. Reviews run inside one terminal or posted only to the repository are invisible to the mesh count. Fifty-three days have no journal event. Costs are not in the journal. The backup contains private text and is not released; the scripts and aggregate tables are.

9. Conclusion

Repowire makes coding-agent sessions on several runtimes into persistent, addressable peers of one mesh, with humans and an orchestrator as peers of the same protocol. Five months of its journal show that the hard part of a persistent mesh is not coordination in the sense the multi-agent literature studies but identity, liveness and delivery: the failures that only exist once an agent can outlive a run. The design decisions the paper defends, daemon-minted process-proved identity with terminal retirement, a lifecycle only on asks, bridges under a transport-neutral router, and lazy repair, are each answerable to an incident class in that record, and the one measurable before-and-after, closure from 87.4% to 98.1%, is the sum of them.

Acknowledgements

[Acknowledge the model vendors whose agents are the peers; the review bots.]

References

Agarwal, A., He, H., and Vasilescu, B. Velocity and warnings: A staggered difference-in-differences study of ai coding agents in open-source repositories. *arXiv preprint arXiv:2601.13597*, 2026.

Anonymous. A survey of LLM agent communication with the model context protocol. *arXiv preprint arXiv:2506.05364*, 2025.

Anonymous. Characterising cross-session memory in LLM

agents. *arXiv preprint arXiv:2606.06448*, 2026a.

Anonymous. InfiAgent: File-centric persistent state for long-horizon agents. In *Findings of the Association for Computational Linguistics*, 2026b.

Anonymous. Survival and churn of agent-authored code. In *Mining Software Repositories*, 2026c. arXiv:2604.00917.

Anthropic. Claude code agent teams. Official documentation, 2026. Accessed 14 October 2026.

Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., and Stoica, I. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems*, 2025. arXiv:2503.13657.

Cursor. Scaling agents with planners and workers, 2026. Engineering post, January 2026. [primary URL].

Ehtesham, A. et al. A survey of agent interoperability protocols: MCP, ACP, A2A, and ANP. *arXiv preprint arXiv:2505.02279*, 2025.

Gao, X., Wang, Y., Yu, Z., Ma, T., and Qu, H. Decisionbench: Evaluating delegation decisions of llm orchestrators. *arXiv preprint arXiv:2605.19099*, 2026.

Hong, S. et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024.

Hu, Q. J. et al. Routerbench: A benchmark for multi-llm routing system. *arXiv preprint arXiv:2403.12031*, 2024.

Khatua, A., Zhu, Y., others, Pei, J., and Yang, D. Cooperbench: Why coding agents cannot be your teammates yet. *arXiv preprint arXiv:2601.13295*, 2026.

Li, G., Hammoud, H., Itani, H., Khizbullin, D., and Ghanem, B. CAMEL: Communicative agents for "mind" exploration of large language model society. In *Advances in Neural Information Processing Systems*, 2023.

Mazloomzadeh, M. et al. An empirical study of agentic pull requests in open source. *arXiv preprint arXiv:2607.21832*, 2026.

Pan, M. Z., Arabzadeh, N., Cogo, F., others, Zaharia, M., and Ellis, K. Measuring agents in production. In *International Conference on Machine Learning*, 2026. arXiv:2512.04123.

Qian, C. et al. ChatDev: Communicative agents for software development. In *Annual Meeting of the Association for Computational Linguistics*, 2024.

Ravishankar, P. When routing benchmarks reward repetition: Evaluating recipient selection in a persistent agent mesh, 2026a. Companion paper, same journal. https://github.com/prassanna-ravishankar/repowire/tree/main/paper/latex.

Ravishankar, P. The vibe bottleneck: AI coding workflow across multi-repo. https://prassanna.io/blog/vibe-bottleneck/, 2026b. Blog post, 23 January 2026.

Ren, S., He, J., Zhang, L., Qian, C., Han, X., Zheng, Y., Li, M., and Zhang, W. Orchbench: Simulation-based evaluation of orchestration plans. *arXiv preprint arXiv:2607.25656*, 2026.

Wang, X. et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

Wu, Q. et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

Yegge, S. Gas town. https://github.com/steveyegge/gastown, 2026. Software and README; v1.0 April 2026.

A. Tool surface over time

Table 6. MCP tools exposed to agents at month end. The Python era counts tool functions; the Go era counts registrations.

month end	tools	added	removed
2025-12	5	`ask_peer, broadcast, list_peers, notify_peer, register_peer`	
2026-01	6	`set_circle, whoami`	`register_peer`
2026-02	7	`kill_peer, spawn_peer`	`set_circle`
2026-03	8	`set_description`	
2026-05	28	`ack, answer, ask, ask_many, ask_many_result, claim_orchestrator_role, job_*` (7), `mark_reviewed, orchestrator_status, review_queue, schedule_*` (5)	`ask_peer`
2026-06	31	`revoke_share, share_session, wait_on_ack`	
2026-08	30		`claim_orchestrator_role`
2026-09	32	`decline, set_name`	
2026-10	33	`restart_peer`	

B. Surfaces by first commit

Table 7. First commit introducing each surface.

surface	date	PR
MCP server with inter-agent tools	2025-12-23	
hosted relay	2025-12-25	
web dashboard	2026-01-23	#2
circles	2026-01-25	#5
peer spawn	2026-01-27	#6
Telegram bot	2026-03-23	#30
Slack bot	2026-03-26	#32
schedules	2026-05-14	#133
SQLite state	2026-05-20	#235
orchestrator role	2026-05-21	#264
jobs	2026-05-25	#310
delivery trace ledger	2026-05-30	#318
fan-out asks	2026-05-31	#334
structured questions	2026-05-31	#335
session sharing	2026-06-11	#360
native Go daemon	2026-08-10	#382
relay-side tool server (remote agents)	2026-10-07	#410
routing decision ledger	2026-10-11	#417
human inbox	2026-10-13	#469

C. Lifecycle events by month

D. Incident examples

One incident per class from the written record, dated by note text.

- **Peer leak or phantom** (2026-05-15): an ack from a retired orchestrator registration, absent from the peer list, arrived as a live request to merge an already-merged pull request.
- **Misroute** (2026-05-14): during co-orchestration, a project peer received three of its last four asks intended for others, and a reply addressed to a suffixed name fell back to broadcast. Fixed by the reconnect identity check.
- **Delivery drop** (2026-05-11): a reply sent on an already-closed ask returned success and was not delivered.

Table 8. Lifecycle events in the journal. Registration events exist only in era B.

month	online	offline	reaped	contradiction	spared by runtime evidence
2026-05	0	0	279	27	0
2026-06	0	318	66	122	493
2026-07	0	132	30	8	16
2026-08	450	96	29	53	100
2026-09	1,900	165	40	55	97
2026-10	1,853	179	41	163	0

- **Daemon restart disruption** (2026-05-10): repeated restarts during installer testing cleared in-memory ask state; stale reminders surfaced and pane kills were refused for pre-restart peers.
- **Config drift** (2026-05-19): the installed CLI lacked a command present in the checkout; a forced reinstall left stale packages.
- **Job or schedule** (2026-06-11): a job fire never armed because the executor's turn carried no marker; the fire stayed delivered.
- **Auth expiry** (2026-06-04): a workspace token rotated to a scopeless state; after re-authentication a stale cache kept calls failing.
- **Other** (2026-05-15): one peer wrote files into another peer's worktree and switched its branch.